Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
División de Ciencias de la Ingeniería

Teoría de Sistemas 2
Laboratorio

Proyecto 1
Sistema de Municipal de Gestión de Residuos
Manual Técnico

Jorge Anibal Bravo Rodríguez
202131782
#3

Introducción **2**
ANÁLISIS DEL SISTEMA **3**
Módulo de Rutas de Recolección 3
Módulo de Puntos Verdes 4
Módulo de Denuncias Ciudadanas 4
Módulo de Reportes 4
Identificación de StakeHolders **5**
1. Usuarios directos 5
2. Entidades Municipales/Gubernamentales 5
3. Otros Interesados 6
Arquitectura del Sistema **7**
1. Capa de Presentación (Views) 7
2. Capa de Lógica de Negocio (Controllers/Services) 7
3. Capa de Acceso a Datos (Models/Eloquent) 7
Tecnologías Utilizadas **8**
Estructura del proyecto **10**
Preparación del Entorno de Ejecución **11**
1. Instalar PHP y extensiones: 11
2. Instalar Composer y Mariadb: 11
3. Clonar y configurar el proyecto 11
4. Crear la base de datos 12
5. Ejecutar migraciones y seeders: 12
6. Habilitar el storage link para subir y consumir imagenes 12
7. Iniciar el servidor 12
Usuarios de Prueba **13**
DESCRIPCIÓN DE LA BASE DE DATOS **14**
Estructura por Módulos 14
Relaciones Clave 15
Algoritmos de Importancia **16**
1. Selección de Ubicación en el mapa 16
2. Generación dinámica de puntos de recolección 17
3. Trazado de Rutas con Leaflet 19
Conclusión **22**

Introducción

El Sistema de Gestión de Residuos y Reciclaje Municipal es un proyecto basado en la web creada para ayudar a la Municipalidad de Quetzaltenango a organizar mejor la recolección de basura, el reciclaje y las denuncias ciudadanas.

La plataforma permite planificar rutas, asignar camiones, controlar puntos verdes y dar seguimiento a cada denuncia, con accesos diferenciados según el rol de cada usuario (administradores, coordinadores, conductores, operadores, auditores y ciudadanos).

El sistema fue desarrollado con una arquitectura de N-Capas que separa la presentación, la lógica de negocio y los datos, facilitando su mantenimiento y crecimiento futuro.

ANÁLISIS DEL SISTEMA

Actualmente el manejo de basura en las municipalidades de Guatemala se hace de forma manual y poco eficiente, provocando que no se aprovechen los camiones de recolección, que la gente no sepa los horarios de recolección de basura y que aparezcan basureros clandestinos en lugares como barrancos. Con este sistema se plantea digitalizar procesos para que las municipalidades tengan mayor control de la basura que se genera y de su recolección y reciclaje.

Para lograr esto se necesita:

- optimizar la logística:
  organizar las rutas de los camiones usando mapas de servicios externos para cubrir las zonas de manera eficiente
- promover el reciclaje:
  darle a la población las herramientas para poder encontrar puntos verdes y que estos tengan el control de las cantidades recolectadas
- limpiar las áreas públicas:
  al crear un canal de comunicación directo con los vecinos que les permita denunciar basureros clandestinos

El sistema será una aplicación web desarrollada bajo una arquitectura de N-Capas que permitirá separar la presentación, la lógica de negocio y el acceso a los datos.

La plataforma permitirá a diferentes tipos de usuarios interactuar según su rol, teniendo permisos específicos para garantizar la seguridad y control.

Las funcionalidades que se integran en el sistema son algunas como:

- Visualización de mapas en rutas
- Registro y control de información relacionada con la recolección de basura
- Registro y seguimiento de denuncias
- Generación de reportes

Divididos en los siguientes módulos:

Módulo de Rutas de Recolección

Este módulo permite:

- Definir zonas con coordenadas geográficas
- Crear rutas de recolección utilizando mapas interactivos
- Asignar días y horarios de recolección
- Registrar camiones y conductores
- Asignar camiones a rutas

- Controlar el estado de cada proceso de recolección
- Registrar basura estimada y basura recolectada
- También permite que los ciudadanos consulten las rutas y horarios de recolección desde un portal público

Módulo de Puntos Verdes

Este módulo gestiona el reciclaje municipal, permitiendo:

- Registrar puntos verdes con ubicación geográfica
- Definir materiales reciclables
- Controlar contenedores y su nivel de llenado
- Registrar entregas realizadas por ciudadanos
- Generar alertas cuando un contenedor alcanza ciertos niveles de llenado

Módulo de Denuncias Ciudadanas

Este módulo permite a los ciudadanos reportar basureros clandestinos, incluyendo:

- Registro de denuncias con ubicación y fotografía
- Clasificación por tamaño del problema
- Control de estados de atención
- Asignación de cuadrillas de limpieza
- Seguimiento del proceso hasta el cierre del caso

Módulo de Reportes

Este módulo permite generar información estadística para la municipalidad, incluyendo reportes de:

- Cantidad de basura recolectada
- Actividad de reciclaje
- Denuncias atendidas y pendientes
- Comparativas por zona y periodo de tiempo

Identificación de StakeHolders

1. Usuarios directos

Stakeholder	**Rol en el Sistema**	**Necesidades Principales**
Administrador Municipal	Acceso total al sistema	Gestionar usuarios, roles, configuración general, ver reportes estratégicos
Coordinador de Rutas	Planifica y asigna rutas	Crear rutas, asignar camiones, monitorear recolección, gestionar denuncias
Operador de Punto Verde	Registra entregas de reciclaje	Registrar entregas, monitorear contenedores, solicitar vaciados
Conductor	Ejecuta rutas de recolección	Ver rutas asignadas, marcar inicio/fin, registrar incidencias y basura recolectada
Auditor	Fiscaliza la información	Ver reportes, exportar datos, validar registros (solo lectura)
Ciudadano	Usuario público/registrado	Reportar denuncias, consultar rutas, ver puntos verdes, dar seguimiento

2. Entidades Municipales/Gubernamentales

Stakeholder	**Relación con el Sistema**	**Interés**
Municipalidad de Quetzaltenango	Entidad propietaria	Eficiencia operativa, transparencia, cumplimiento normativo

Dirección de Gestión Ambiental	Supervisa el sistema	Reducción de basura, aumento de reciclaje, control de denuncias
Tesorería Municipal	Recibe reportes	Datos para presupuesto y planificación
Consejo Municipal	Toma decisiones	Estadísticas generales, impacto del servicio

3. Otros Interesados

Stakeholder	**Interés**
Ministerio de Ambiente	Cumplimiento de normativas ambientales nacionales
Universidades (USAC, URL, etc.)	Posibles estudios de caso, prácticas profesionales
Empresas Recicladoras	Posibles convenios para compra de material reciclado
Turismo	Que la ciudad esté limpia para visitantes

Arquitectura del Sistema

El sistema implementa una Arquitectura N-Capas que separa claramente las responsabilidades en tres niveles:

1. Capa de Presentación (Views)

- Tecnología: Laravel Blade, JavaScript, Bootstrap
- Ubicación: resources/views/
- Función: Interfaces de usuario organizadas por rol (administrador, coordinador de rutas, conductor, ciudadano, auditor y encargado de punto verde).
- Características: Vistas limpias sin lógica de negocio, solo presentación de datos

2. Capa de Lógica de Negocio (Controllers/Services)

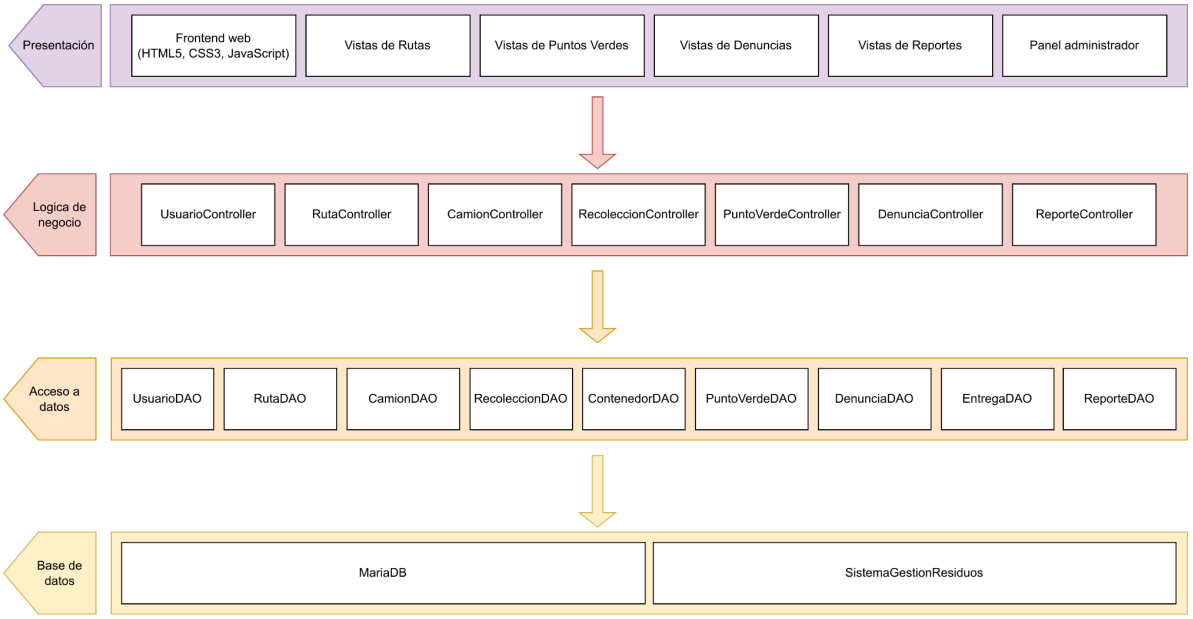
- Tecnología: PHP 8.2.30, Laravel
- Ubicación: app/Http/Controllers/ y app/Services/
- Función:
    - Controllers: Reciben peticiones, validan datos y coordinan respuestas
    - Services: Contienen la lógica compleja y reglas de negocio reutilizables, de esta manera los servicios pueden ser usados por múltiples controladores

3. Capa de Acceso a Datos (Models/Eloquent)

- Tecnología: ORM para MariaDB
- Ubicación: app/Models/
- Función: Representación de tablas como objetos PHP y sus relaciones entre entidades
- Características: Migraciones para control de versiones de BD, seeders para datos de prueba

En base a esta arquitectura el flujo de datos es el siguiente:

1. Petición web
2. Rutas en web.php
3. Middleware con roles
4. Controller
5. Service
6. Model
7. Base de datos



Tecnologías Utilizadas

- **PHP 8.2.30:**
  Lenguaje en el que se desarrolló todo el backend.
- **Laravel 11:**
  Framework de PHP sobre el que se construyó el backend.
- **MariaDB 11.8.3:**
  Gestor de base de datos relacional
- **Apache 2.4.66:**
  Servidor web
- **Composer:**
  Gestor de dependencias para PHP
- **Bootstrap:**

Framework CSS para interfaces

- **Leaflet:**

  Api para manejo de mapas web

- **Chart js:**

  Biblioteca para gráficos web

- **Git:**

  Sistema de control de versiones

- **Debian 13:**

  Sistema operativo sobre el que se construyó el sistema

Estructura del proyecto

```
gestion-residuos-municipales/
├── app/
│   ├── Http/
│   │   ├── Controllers/    Controladores por rol (Admin, Coordinador, etc.)
│   │   └── Middleware/     RoleMiddleware.php (control de acceso por rol)
│   ├── Models/             Modelos Eloquent (Zona, Ruta, Denuncia, etc.)
│   └── Services/           Lógica de negocio reutilizable
├── bootstrap/              Archivos de inicio de Laravel
├── config/                 Configuración de la aplicación
├── database/
│   ├── migrations/         Estructura de tablas
│   └── seeders/            Datos de prueba para cada tabla
├── public/                 Punto de entrada y archivos públicos como bootstrap local
├── resources/
│   └── views/              Vistas Blade organizadas por rol
│       ├── admin/          Vistas para administradores
│       ├── auditor/        Vistas para auditores
│       ├── ciudadano/      Vistas para ciudadanos
│       ├── conductor/      Vistas para conductores
│       ├── coordinator/    Vistas para coordinadores
│       ├── layouts/        Plantillas base para los roles del sistema
│       ├── operador/       Vistas para operadores
│       └── public/         Vistas públicas sin autenticación
├── routes/
│   └── web.php             Todas las rutas de la aplicación
└── storage/                Archivos subidos
```

Preparación del Entorno de Ejecución

1. Instalar PHP y extensiones:

```
# Agregar repositorio de PHP (Debian/Ubuntu)
sudo apt update
sudo apt install -y curl wget git unzip
sudo wget -O /etc/apt/trusted.gpg.d/php.gpg
https://packages.sury.org/php/apt.gpg
echo "deb https://packages.sury.org/php/ $(lsb_release -sc) main" | sudo
tee /etc/apt/sources.list.d/php.list
sudo apt update

# Instalar PHP 8.2 y extensiones
sudo apt install -y php8.2 php8.2-cli php8.2-common php8.2-curl
php8.2-mbstring php8.2-mysql php8.2-xml php8.2-zip php8.2-bcmath
php8.2-tokenizer php8.2-intl php8.2-gd
```

2. Instalar Composer y Mariadb:

```
# Instalar Composer
curl -sS https://getcomposer.org/installer | php
sudo mv composer.phar /usr/local/bin/composer

# Instalar MySQL/MariaDB
sudo apt install -y mariadb-server
sudo systemctl start mariadb
```

3. Clonar y configurar el proyecto

```
# Clonar repositorio
git clone git@github.com:Jbrod5/gestion-residuos-municipales.git
cd gestion-residuos-municipales

# Instalar dependencias
composer install

# Configurar entorno
cp .env.example .env
php artisan key:generate
```

4. Crear la base de datos

```
# Crear base de datos
mysql -u root -p -e "CREATE DATABASE gestion_residuos CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;"

# Editar archivo .env para agregar DB_DATABASE, DB_USERNAME, DB_PASSWORD
```

5. Ejecutar migraciones y seeders:

```
php artisan migrate --seed
php artisan storage:link

# O usar el script de bash db_reset
./db_reset.sh
```

6. Habilitar el storage link para subir y consumir imagenes

```
php artisan storage:link
```

7. Iniciar el servidor

```
php artisan serve
```

Usuarios de Prueba

- Admin:
  - admin@muni.com
  - pass123
- Coordinador:
  - coordinador@muni.com
  - pass123
- Operador:
  - operador@muni.com
  - pass123
- Auditor:
  - auditor@muni.com
  - pass123
- Conductor:
  - conductor@muni.com
  - pass123
- Ciudadano:
  - ciudadano@correo.com
  - pass123

DESCRIPCIÓN DE LA BASE DE DATOS

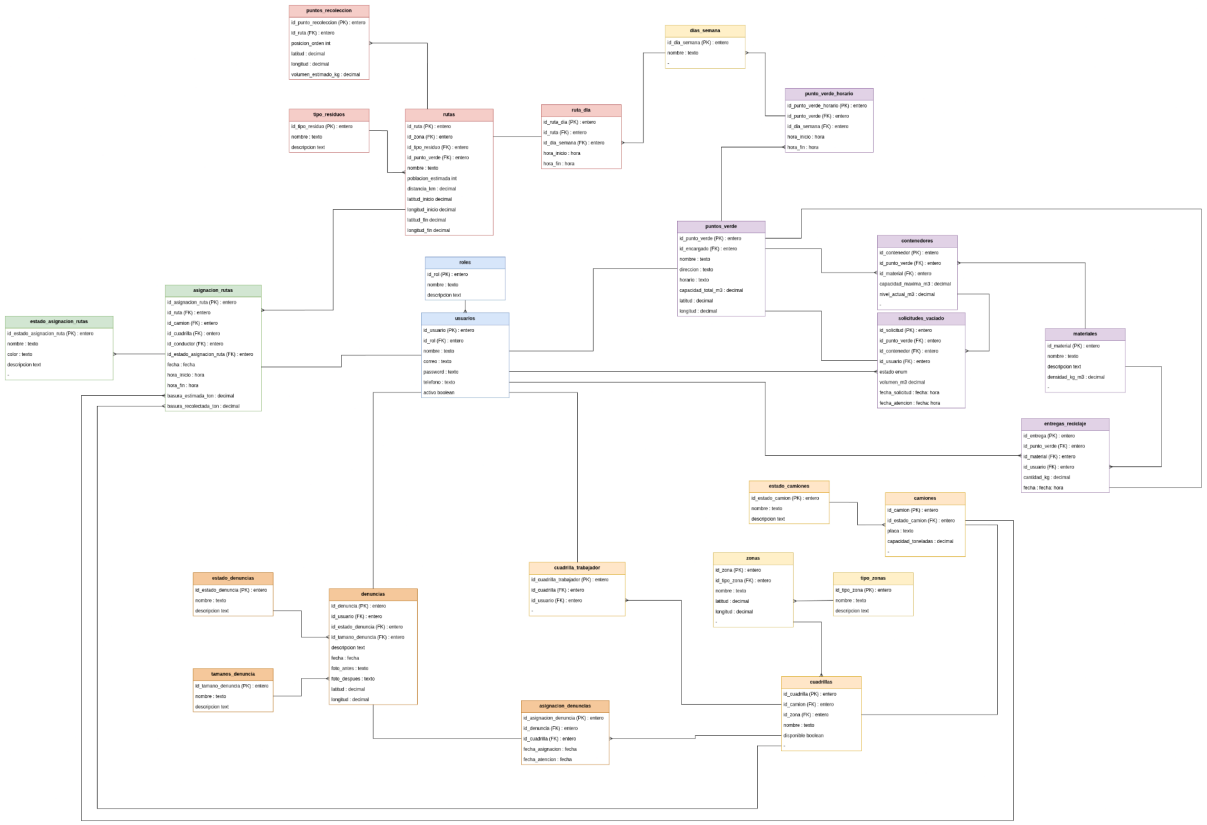
La base de datos del sistema está diseñada bajo el motor MariaDB y fue manejada desde las migraciones de Laravel.

Estructura por Módulos

Módulo	**Tablas Principales**	**Propósito**
Seguridad	roles, usuarios, sessions	Autenticación y control de acceso
Rutas	rutas, zonas, tipo_zonas, ruta_dia, ruta_trayectorias, puntos_recoleccion	Gestión de recorridos de recolección
Logística	camiones, estado_camiones, cuadrillas, cuadrilla_trabajador	Flota vehicular y equipos de trabajo
Asignaciones	asignacion_rutas, estado_asignacion_rutas	Programación de rutas a camiones
Reciclaje	puntos_verde, materiales, contenedores, entregas_reciclaje, solicitudes_vaciado	Gestión de puntos verdes y reciclaje
Denuncias	denuncias, estado_denuncias, tamanos_denuncia, asignacion_denuncias	Reportes ciudadanos y su atención

Relaciones Clave

- usuarios se relaciona con roles (pertenece a un rol)
- asignacion_rutas es la tabla central que relaciona rutas, camiones, cuadrillas, conductores y estados
- puntos_verde se relaciona con usuarios (encargados) y contenedores (materiales)
- denuncias se relaciona con asignacion_denuncias para vincular con cuadrillas



Algoritmos de Importancia

1. Selección de Ubicación en el mapa

Su propósito es Permitir al administrador seleccionar la ubicación exacta de un punto verde haciendo clic en el mapa de la siguiente manera:

- Inicializar mapa centrado en Guatemala
- Escuchar clics en el mapa
- Al hacer clic:
    a. Eliminar marcador anterior (si existe)
    b. Crear nuevo marcador en la posición clickeada
    c. Actualizar campos ocultos de latitud y longitud
- Enviar coordenadas al servidor al guardar

Explicaciónde código:

```
// Inicializar mapa
var map = L.map('map').setView([14.6349, -90.5069], 13);
L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png').addTo(map
);

var marker; // variable para el marcador actual

// Evento click
map.on('click', function (e) {
    // Eliminar marcador anterior si existe
    if (marker) map.removeLayer(marker);

    // Crear nuevo marcador
    marker = L.marker(e.latlng).addTo(map);

    // Actualizar campos ocultos (6 decimales de precisión)
    document.getElementById('lat').value = e.latlng.lat.toFixed(6);
    document.getElementById('lng').value = e.latlng.lng.toFixed(6);
});
```

2. Generación dinámica de puntos de recolección

Su propósito es generar automáticamente entre 15 y 30 puntos de recolección a lo largo de la ruta trazada, con pesos variables según el tipo de zona de la siguiente manera:

- Obtener la trayectoria completa de la ruta (puntos trazados)
- Determinar número aleatorio de puntos (15-30)
- Determinar si la zona es de alta carga (comercial/industrial)
- Para cada punto a generar:
    - Seleccionar un segmento aleatorio entre dos puntos de la trayectoria
    - Interpolar posición entre esos dos puntos
    - Asignar peso según tipo de zona
    - Guardar en base de datos
- Retornar peso total estimado

Explicación de código:

```
private function generarPuntosRecoleccion(Ruta $ruta)
{
    // Obtener la trayectoria ordenada
    $trayectorias = $ruta->trayectorias()->orderBy('orden')->get();

    // Determinar cantidad aleatoria de puntos (15-30)
    $numPuntos = rand(15, 30);
    $totalTrayectorias = $trayectorias->count();
    $sumaPeso = 0;


// Determinar si es zona de alta carga
    $idTipoZona = $ruta->zona->id_tipo_zona;
    $esAltaCarga = in_array($idTipoZona, [6, 7]); // IDs de comercial/industrial


   for ($i = 0; $i < $numPuntos; $i++) {
        if ($totalTrayectorias > 1) {
            // Seleccionar segmento aleatorio entre dos puntos consecutivos
            $idx = rand(0, $totalTrayectorias - 2);
            $p1 = $trayectorias[$idx];
            $p2 = $trayectorias[$idx + 1];

            // INTERPOLACIÓN LINEAL: P = P1 + t * (P2 - P1)
            $t = rand(0, 1000000) / 1000000; // factor aleatorio entre 0 y 1
```

```php
        $lat = $p1->latitud + $t * ($p2->latitud - $p1->latitud);
        $lng = $p1->longitud + $t * ($p2->longitud - $p1->longitud);
    } else {
        // Si solo hay un punto, usar ese
        $lat = $trayectorias[0]->latitud;
        $lng = $trayectorias[0]->longitud;
    }




    // Asignar peso según tipo de zona
    $peso = $esAltaCarga ? rand(500, 2000) : rand(50, 500);
    $sumaPeso += $peso;

    // Guardar punto en base de datos
    PuntoRecoleccion::create([
        'id_ruta' => $ruta->id_ruta,
        'posicion_orden' => $i + 1,
        'latitud' => $lat,
        'longitud' => $lng,
        'volumen_estimado_kg' => $peso,
    ]);
}

return $sumaPeso;
```

3. Trazado de Rutas con Leaflet

Su propósito es permitir al coordinador trazar visualmente el recorrido de una ruta de recolección haciendo clics en el mapa, almacenando las coordenadas para generar puntos de recolección automáticos de la siguiente manera:

- Inicializar mapa con Leaflet y centrarlo en Quetzaltenango
- Escuchar clics del usuario en el mapa
- Por cada clic:
    a. Guardar coordenada en array
    b. Dibujar marcador visual
    c. Actualizar línea de ruta
- Calcular distancia total entre todos los puntos
- Habilitar botón guardar cuando haya al menos 2 puntos
- Al guardar: enviar array de coordenadas al servidor

Explicación de código:

```
// 1. INICIALIZACIÓN DEL MAPA
var map = L.map('map').setView([14.6349, -90.5069], 13);
```

- `L.map('map')`: Crea el mapa en el elemento con id="map"
- `.setView([14.6349, -90.5069], 13)`: Centra en Quetzaltenango con zoom nivel 13

```
// 2. CAPA BASE DE OPENSTREETMAP
L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
    attribution: '&copy; OpenStreetMap contributors'
}).addTo(map);
```

- Añade los mosaicos (tiles) de OpenStreetMap
- `{s}`, `{z}`, `{x}`, `{y}` son reemplazados automáticamente por Leaflet

```
// 3. VARIABLES DE ESTADO
var pathPoints = [];            // Array de coordenadas [lat, lng]
var polyline = L.polyline([], { color: '#ffc107', weight: 5 }).addTo(map);

var markers = [];               // Array de marcadores
```

- `pathPoints`: Almacena las coordenadas de cada clic
- `polyline`: Línea amarilla que conecta los puntos
- `markers`: Guarda referencias a los marcadores para poder eliminarlos

```
// 4. EVENTO CLICK EN EL MAPA
map.on('click', function (e) {
    const point = [e.latlng.lat, e.latlng.lng];
    pathPoints.push(point);

    // Dibujar marcador circular
    const marker = L.circleMarker(e.latlng, {
        radius: 4,
        fillColor: "#212529",
        color: "#ffc107"
    }).addTo(map);
    markers.push(marker);

    // Actualizar línea
    polyline.addLatLng(e.latlng);

    // Calcular distancia
    actualizarMetricas();
});
```

- Cada clic: guarda coordenada, dibuja marcador, actualiza línea y recalcula

```
// 5. CÁLCULO DE DISTANCIA
function actualizarMetricas() {
    if (pathPoints.length < 2) {
        document.getElementById('distancia_km').value = "0.00";
        document.getElementById('btnGuardar').disabled = true;
        return;
    }

    let totalDist = 0;
    for (let i = 1; i < pathPoints.length; i++) {
        totalDist += map.distance(pathPoints[i-1], pathPoints[i]);
    }

    const distKm = (totalDist / 1000).toFixed(2);
    document.getElementById('distancia_km').value = distKm;
```

```
    document.getElementById('camino_coordenadas').value =
JSON.stringify(pathPoints);
    document.getElementById('btnGuardar').disabled = false;

}
```

- `map.distance()`: Método de Leaflet que calcula distancia exacta entre dos puntos geográficos
- Convierte de metros a kilómetros
- Guarda el array como JSON en campo para enviar al servidor

```
// 6. LIMPIAR MAPA
window.limpiarMapa = function () {
    pathPoints = [];
    polyline.setLatLngs([]);
    markers.forEach(m => map.removeLayer(m));
    markers = [];
    actualizarMetricas();

};
```

- Reinicia todo: vacía array, limpia línea, elimina marcadores

Conclusión

El sistema cumple con todos los objetivos planteados, mejorando la eficiencia en la recolección de desechos y la gestión del reciclaje en Quetzaltenango. Gracias a sus mapas interactivos, el control de contenedores en tiempo real y el seguimiento completo de denuncias, la municipalidad puede ofrecer un servicio más ordenado y transparente.

Los diferentes roles de usuario garantizan que cada persona haga lo que le corresponde, y los reportes permiten tomar decisiones informadas.
En resumen, el sistema contribuye a tener una ciudad más limpia y a facilitar la participación ciudadana en el cuidado del medio ambiente.